🖥️ Introduction to HPC

SLURM Workshop — Lab Worksheet

RSU HPC AI Data Center • 2026

ชื่อ - นามสกุล		วันที่

Username (HPC)	

🎯 **วัตถุประสงค์การเรียนรู้**

- เข้าใจโครงสร้างของ HPC Cluster และความแตกต่างระหว่าง Login Node vs Compute Node
- สามารถ SSH เข้าสู่ระบบและ navigate ด้วย command line ได้
- เข้าใจ Lmod module system และโหลด software ที่ต้องการได้
- ส่ง SLURM job แบบ interactive (srun) และแบบ batch (sbatch) ได้
- ใช้ JupyterHub บน HPC และรัน Python script ผ่าน SLURM ได้
- Monitor job status และดู output/error log ได้

ส่วนที่ 1 — HPC Basics & SSH

1 SSH เข้าสู่ระบบ rsu-login

เปิด Terminal แล้วพิมพ์คำสั่งต่อไปนี้ แทนที่ username ด้วย username ของคุณ

```
# เชื่อมต่อ Login Node
ssh <username>@171.102.216.177

# ตรวจสอบ hostname
hostname
whoami
```

💡 **Login Node vs Compute Node**

- rsu-login = ที่สำหรับเตรียมงาน เขียน script ไม่ควรรัน program หนักๆ โดยตรง
- rsu-training = Compute Node มี GPU H200 สำหรับ run actual jobs
- SLURM จะ allocate resource และส่ง job ไปยัง Compute Node ให้อัตโนมัติ

*สิ่งที่เห็นหลังจาก **login** สำเร็จ **(hostname, kernel version** ฯลฯ**)***

2 สำรวจ filesystem

```
# ดู directory ปัจจุบัน
pwd

# ดูรายการไฟล์ พร้อม size และ permission
ls -lh ~

# ดู disk quota ของตัวเอง
df -h $HOME

# ดู storage shared
ls /cm/shared/
```

Home directory** ของคุณอยู่ที่ **path** ไหน**? (**รันคำสั่ง **pwd)

*พื้นที่ **/cm/shared/** มีอะไรบ้าง **(list 3-5** รายการ**)***

Rangsit University

ส่วนที่ 2 — Miniconda

1 **SSH เข้าสู่ระบบ rsu-login**

ทำผ่าน Terminal

เปิด Terminal แล้ว SSH เข้า cluster ก่อน

```
$ ssh <username>@171.102.216.177
```

Lab 1-1: เปิดใช้งาน Miniconda (Shared)

📌 **Miniconda Path**

Miniconda บน cluster ติดตั้งแบบ shared ที่: /cm/shared/apps/miniconda3 ทุก user ใช้ร่วมกันได้ ไม่ต้องติดตั้งเอง

1 **เปิดใช้งาน conda ใน shell session**
ตรวจสอบ conda ภายใน environment ของตนเอง

```
$ source /cm/shared/apps/miniconda3/etc/profile.d/conda.sh
$ conda --version
```

📝 *conda version ที่ได้:*

2 **ดู environment ที่มีอยู่**
ดูทั้ง shared env และ env ส่วนตัว

```
$ conda env list
```

📝 *จด environment ที่เห็น (เขียนอย่างน้อย 2 env):*

3 Activate environment สำเร็จรูปที่ผู้สอนเตรียมไว้

```
$ conda activate torch_env
$ python --version
$ python -c "import torch; print(torch.__version__)"
```

📝 *Python version และ PyTorch version ที่ได้:*

💡 **Tips — เพิ่มใน ~/.bashrc ให้ source อัตโนมัติ**

รัน /cm/shared/apps/miniconda3/bin/conda init หลังจากนี้ไม่ต้อง source ทุกครั้งที่ login

Lab 1-2: สร้าง Conda Environment ของตัวเอง

1 สร้าง environment ใหม่พร้อม Python 3.11

```
$ conda create -n env1 python=3.11 -y
$ conda config --append channels conda-forge
$ conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
$ conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
$ conda create -n env1 python=3.11 -y
$ conda activate env1
$ conda install -y -c file:///cm/shared/apps/jupyter/current/share/conda-repo cm-jupyter-eg-kernel-wlm
$ python --version
```

2 ติดตั้ง packages ใน environment

```
$ conda install numpy pandas matplotlib -y
$ pip install torch --index-url https://download.pytorch.org/whl/cu121
$ python -c "import torch, numpy; print('OK', torch.__version__)"
```

3 Export และ Deactivate

```
$ conda env export > myenv.yml   # backup environment
$ conda deactivate
$ conda env list                 # ดู env ทั้งหมด
```

📝 ***Python version** และ **PyTorch version** ที่ได้:*

ส่วนที่ 3 — Lmod Module System

Lmod ใช้จัดการ software environment บน HPC ป้องกัน version conflict

3 โหลด **Module** ที่ต้องการ

```
# ดู module ทั้งหมดที่มี
module avail

# ดู module ที่โหลดอยู่ตอนนี้
module list

# โหลด CUDA 13.0 Toolkit
module load cuda13.0/toolkit


# ตรวจสอบ
nvcc -V
python --version

# Unload module
module unload cuda13.0/toolkit

# Reset ทั้งหมด
module purge
```

***CUDA version** ที่ได้จาก **nvcc -V** เวอร์ชั่นอะไร?*

⚠️ **ข้อควรระวัง**

- module load ใน interactive session ไม่ carry over ไปยัง batch job
- ต้องใส่ module load ไว้ใน SLURM script ด้วยทุกครั้ง

- conda activate ต้องมี source conda.sh ก่อน ใน batch job

ส่วนที่ 4 — SLURM Job Submission

4.1 SLURM คืออะไร?

SLURM (Simple Linux Utility for Resource Management) คือ job scheduler ที่จัดการคิวและ allocate GPU/CPU ให้กับ user แต่ละคนอย่างยุติธรรม

คำสั่ง	ความหมาย	ตัวอย่าง
`sbatch script.sh`	ส่ง batch job	`sbatch train_model.sh`
`srun --pty bash`	Interactive job	`srun --gres=gpu:1 -c 2 \ -mem=4G --pty bash`
`squeue -u $USER`	ดู job ของตัวเอง	`squeue -u test01`
`scancel <JOBID>`	ยกเลิก job	`scancel 970`
`sinfo`	ดูสถานะ cluster	`sinfo -N -l`
`sacct -j <ID>`	ดู job history	`sacct -j 970 --format=...`

1 ดูสถานะ cluster และ partitions

```
$ sinfo
```

Partition	State	# Nodes	Time Limit

2 ดู resource limit ของตัวเอง

```
$ sacctmgr show assoc format=account,user%15,GrpTRES%80 where user=$USER
$ squeue -u $USER
```

📝 จด ***resource limit*** *ของตัวเอง* ***(CPU / Memory / GPU):***

3 Interactive Job — สำรวจ GPU

Interactive job ใช้สำหรับ test, debug, หรือ explore environment ก่อนสร้าง script

```
# ขอ interactive shell พร้อม GPU 1 ตัว (MIG 1g.18gb)
srun --gres=gpu:1g.18gb:1 --cpus-per-task=2 --mem=8G --pty bash

# เมื่อได้ shell บน Compute Node แล้ว:
hostname            # ควรเห็น rsu-training
nvidia-smi          # ดู GPU ที่ได้รับ
nvidia-smi --query-gpu=name,memory.total --format=csv

# โหลด CUDA
module load cuda13.0/toolkit
nvcc -V

# ออกจาก Interactive job
exit
```

GPU ที่ได้จาก MIG 1g.18gb มี VRAM กี่ GB? (จาก nvidia-smi)

Rangsit University

4 Batch Job — Python Hello World

สร้างไฟล์ script ด้วย nano หรือ vi

```
# สร้าง Python script
cat > hello_gpu.py << 'EOF'
import torch
print(f'PyTorch: {torch.__version__}')
print(f'CUDA available: {torch.cuda.is_available()}')
if torch.cuda.is_available():
    print(f'GPU: {torch.cuda.get_device_name(0)}')
    x = torch.rand(1000, 1000).cuda()
    print(f'Tensor on GPU: {x.shape}')
EOF

# สร้าง SLURM script
cat > run_hello.sh << 'EOF'
#!/bin/bash
#SBATCH --job-name=hello-gpu
#SBATCH --gres=gpu:1g.18gb:1
#SBATCH --cpus-per-task=2
#SBATCH --mem=8G
#SBATCH --time=00:05:00
#SBATCH --output=hello-%j.out

#module load miniconda3
source /cm/shared/apps/miniconda3/etc/profile.d/conda.sh
conda activate torch-env

python hello_gpu.py
EOF

# Submit job
sbatch run_hello.sh

# Monitor
squeue -u $USER
tail -f hello-<JOBID>.out
```

***JOBID** ที่ได้จาก **sbatch** คืออะไร และ **status** ของ **job** คืออะไร (R/PD)?*

***Output** ที่ได้จาก **hello_gpu.py** มีข้อความอะไรบ้าง?*

ส่วนที่ 5 — JupyterHub บน HPC

JupyterHub ให้ใช้งาน Jupyter Notebook ผ่าน browser โดย kernel รันบน Compute Node ผ่าน SLURM

1 เข้าใช้งาน JupyterHub

1	เปิด browser ไปที่ http://171.102.216.177:8000
2	Login ด้วย username/password เดียวกับ SSH
3	สร้าง Kernel Template ที่ชื่อ Python Conda kernel on SLURM กำหนด Conda Env, CPU, Memory, GPU, Display name
4	เลือก Kernel ที่สร้างไว้จากหน้า Launcher และรอ SLURM allocate resources (อาจใช้เวลา 1-2 นาที)
5	เมื่อ Jupyter เปิดขึ้นมาแล้ว ให้รอสถานะจาก Initializing จนขึ้น idle แล้วจึงรันโปรแกรม

2 รัน Python Notebook

พิมพ์โค้ดต่อไปนี้ใน Notebook แล้วกด Shift+Enter เพื่อรัน

```
# Cell 1: ตรวจสอบ GPU
import torch
print('PyTorch:', torch.__version__)
print('GPU available:', torch.cuda.is_available())
print('GPU name:', torch.cuda.get_device_name(0))

# Cell 2: Matrix computation บน GPU
a = torch.rand(2000, 2000).cuda()
b = torch.rand(2000, 2000).cuda()
c = torch.matmul(a, b)
print('Result shape:', c.shape)
print('Device:', c.device)

# Cell 3: Monitor GPU memory
print('GPU Memory Allocated:', torch.cuda.memory_allocated(0)/1024**2, 'MB')
print('GPU Memory Reserved:', torch.cuda.memory_reserved(0)/1024**2, 'MB')
```

***GPU Memory** ที่ถูกใช้งาน **(Allocated)** หลังรัน **Matrix computation** คือ?*

6 Shutdown Kernel

ส่วนที่ 6 — Checklist สรุป

ทำเครื่องหมาย ☑ ในช่องที่ทำสำเร็จแล้ว

☐	SSH เข้า rsu-login สำเร็จ
☐	ตรวจสอบ home directory และ quota ได้
☐	โหลด module cuda13.0 และตรวจสอบด้วย nvcc -V
☐	ส่ง interactive job ด้วย srun และ nvidia-smi บน Compute Node
☐	สร้าง SLURM batch script และ sbatch สำเร็จ
☐	ดู output file จาก batch job
☐	เข้า JupyterHub ได้ผ่าน browser
☐	รัน Python + PyTorch บน GPU ใน Notebook สำเร็จ

📝 คำถาม **Reflection**

1. ความแตกต่างระหว่าง srun และ sbatch คืออะไร? ควรใช้อันไหนในสถานการณ์แบบไหน?

2. ถ้า job ค้างใน queue นานผิดปกติ จะ debug อย่างไร?

Rangsiman